Tugas Pendahuluan Praktikum Dasar Kecerdasan Artifisial

Modul 6 : Implementasi Breadth-First Search (BFS)

INSTALLASI LIBRARY MK DKA
Jika belum terinstal, jalankan perintah berikut pada terminal atau notebook:

```
pip install jupyter ipykernel numpy matplotlib networkx sympy scikit-fuzzy scipy
```

Petunjuk Umum

1. **Jangan ubah struktur kode yang diberikan, cukup isi bagian yang kosong.**
2. **Jalankan setiap sel satu per satu agar tidak ada error.**
3. **Gunakan modul praktikum atau dokumentasi NetworkX (`https://networkx.org/documentation/stable/`) jika membutuhkan referensi tambahan.**
4. **Kerjakan soal yang ada secara berurutan, karena soal saling berkesinambungan.**

In [1]:
```
print("Muhammad Alvin Faa'iz")  # Tuliskan nama lengkap Anda
print("103012400229")  # Tuliskan NIM Anda
print("IF-48-11")  # Tuliskan kelas Anda
print("VVN") # Tuliskan kode asprak anda
```

```
Muhammad Alvin Faa'iz
103012400229
IF-48-11
VVN
```

Pengantar Breadth-First Search (BFS)

Breadth-First Search (BFS) adalah algoritma pencarian yang digunakan dalam graf dan pohon. BFS menjelajahi node secara level-per-level, mulai dari simpul awal dan mengunjungi semua simpul tetangga terlebih dahulu sebelum melanjutkan ke level berikutnya.

In [2]:
```
import networkx as nx # Import library networkX
import matplotlib.pyplot as plt # Import library matplotlib.pyplot
```

Contoh fungsi pendukung untuk mencetak graf

!! Tidak perlu dimodifikasi !!

In [3]:
```
# Support function to print the graph
def show_graph(G, pos=None, title='') :
    # If position is not specified from the parameter it will generate a new positi
    if pos == None :
        # The pos variable that stores the coordinates of the node placement
        pos = nx.spring_layout(G)

    # Plot the graph G given in the parameter
    nx.draw(G, pos, with_labels=True, node_color='red', node_size=2500,
            font_color="white", font_weight="bold", width=5)

    # Retrieve each weight of each edge in graph G
    edge_labels = nx.get_edge_attributes(G, "weight")

    # Draw the weight of the edge in the visualization
    nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_color='blue'
                                 font_weight="bold", font_size=12)

    # Add margins on each side
    plt.margins(0.2)
    # Add a title as given by the parameter
    plt.title(title)
    # Display the graph visualization on the terminal
    plt.show()
```

Soal 1

Buatlah sebuah graf tidak berarah sederhana dengan 6 simpul: A, B, C, D, E, F. Tambahkan edge sebagai berikut:

- A - B
- A - C
- B - D
- B - E
- C - F

Visualisasikan graf tersebut menggunakan `matplotlib` dan `networkx.draw()`.

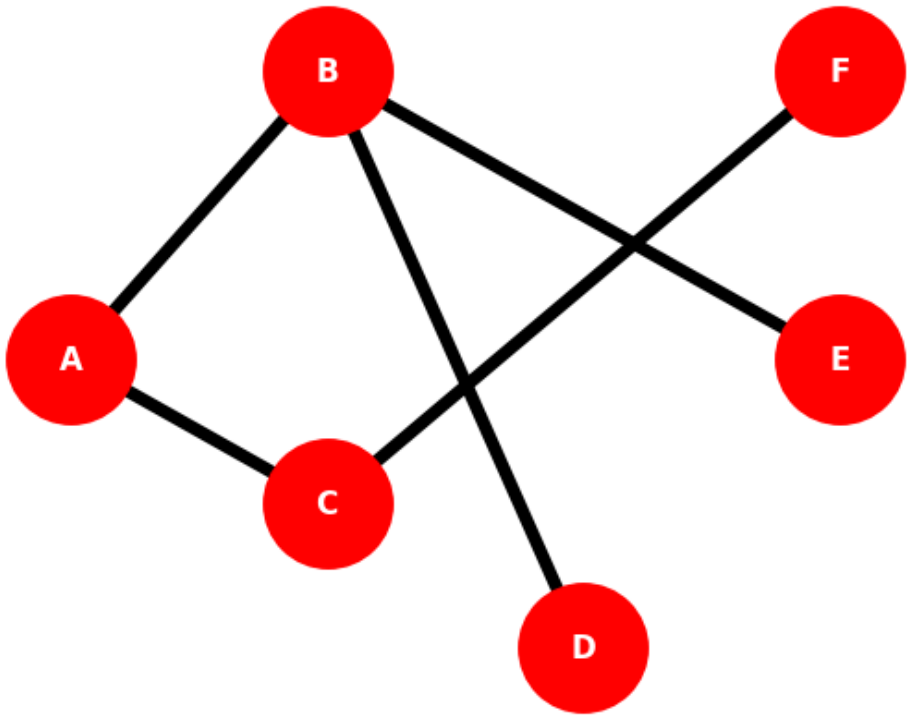
!! Tidak usah dimodifikasi !!

In [4]:
```
# Posisi untuk undirected graph agar sesuai dengan contoh soal
pos = {
    "A": (0, 1),
    "B": (1, 3),
    "C": (1, 0),
    "D": (2, -1),
    "E": (3, 1),
    "F": (3, 3)
}
```

In [5]:
```
# Inisialisasi graf tidak berarah
G = nx.Graph()

# Tambahkan edge di sini
edges = [("A", "B"), ("A","C"), ("B","D"), ("B","E"), ("C","F")]
G.add_edges_from(edges)

# Tampilkan graf
show_graph(G, pos=pos)
```



Contoh *output*:

tp5_scatter

Soal 2

Gunakan fungsi `nx.bfs_edges()` untuk melakukan pencarian BFS dari simpul A dan C. Cetak urutan traversal BFS dalam bentuk edge.

In [18]:
```
# Implementasikan BFS dan cetak edge hasil traversal
def pencarian_simpul_BFS(G, source):
    bfs_edges = list(nx.bfs_edges(G, source))
    print("Hasil BFS transversal simpul", source, ":")
    for u, v in bfs_edges:
        print(u, "->", v)
```

In [21]:
```
masukan = input("Masukkan simpul awal: ")
pencarian_simpul_BFS(G, masukan)
```

```
Hasil BFS transversal simpul A
A -> B
A -> C
B -> D
B -> E
C -> F
```

In [22]:
```
masukan = input("Masukkan simpul awal: ")
pencarian_simpul_BFS(G, masukan)
```

```
Hasil BFS transversal simpul C
C -> A
C -> F
A -> B
B -> D
B -> E
```

Test case Program:

input	output
Masukan simpul awal: `A`	Hasil BFS traversal simpul A: A -> B A -> C B -> D B -> E C -> F
Masukan simpul awal: `C`	Hasil BFS traversal simpul C: C -> A C -> F A -> B B -> D B -> E

Soal 3

Ubah graf sebelumnya menjadi **graf berarah** dan ulangi proses BFS dari simpul A. Bandingkan hasilnya dengan soal sebelumnya.

!! Tidak usah dimodifikasi !!

In [24]:
```
# Inisialisasi graf berarah
DG = nx.DiGraph()
# tambahkan edges ke var graf berarah dari edges yang ada pada G
DG.add_edges_from(G.edges)
```

In [30]:
```
def pencarian_simpul_BFS_directed(source):
    bfs_edges_directed = list(nx.bfs_edges(DG, source))
    print("BFS pada Directed Graph simpul ", source , ":")
```

```
    for u, v in bfs_edges_directed:
        print(u, "->", v)
```

In [31]:
```
masukan = input("Masukan simpul awal: ")
pencarian_simpul_BFS_directed(masukan)
```

```
BFS pada Directed Graph simpul  A :
A -> B
A -> C
B -> D
B -> E
C -> F
```

In [32]:
```
masukan = input("Masukan simpul awal: ")
pencarian_simpul_BFS_directed(masukan)
```

```
BFS pada Directed Graph simpul  C :
C -> F
```

Test case Program:

input	output
Masukan simpul awal: `A`	BFS pada Directed Graph simpul A: A -> B A -> C B -> D B -> E C -> F
Masukan simpul awal: `C`	BFS pada Directed Graph simpul C: C -> F

In [ ]: